Distributed Trip-Level Fuel Consumption Forecasting

from raw vehicle telemetry to trip-level fuel consumption prediction

Big Data Final Project

Team 15: Mikhail, Ilyas, Andrei, Kamilya

Project and Objective

Problem: Raw vehicle telemetry is sensor-level: one trip consists of many timestamped rows. It cannot be directly used for trip-level ML.

Objective: Build a distributed pipeline that converts raw telemetry into clean trip-level features, analyzes fuel-consumption patterns, trains forecasting models, and presents the results in Superset.

Why it matters: Fleet operators need to understand which trips and conditions lead to inefficient fuel consumption.

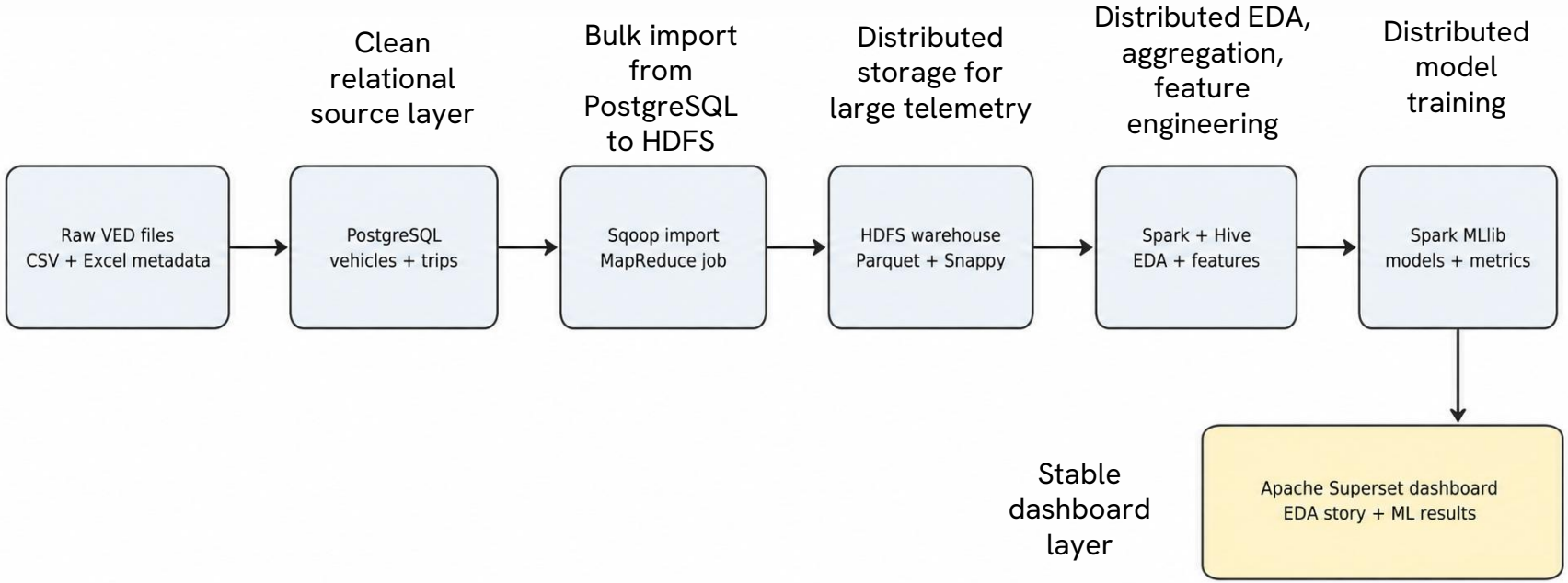
Dataset Characteristics

Metric	Value
Raw telemetry rows	22.4M
Vehicles	384
Trips before filtering	~32.5k
Trainable trips	~25.2k
Target	fuel_l_per_100km

Why distributed processing?

Because the raw telemetry table is large, sensor-level, and repeatedly processed for profiling, EDA, feature engineering, and ML

Pipeline Architecture



Stage 1: Data Collection and Ingestion

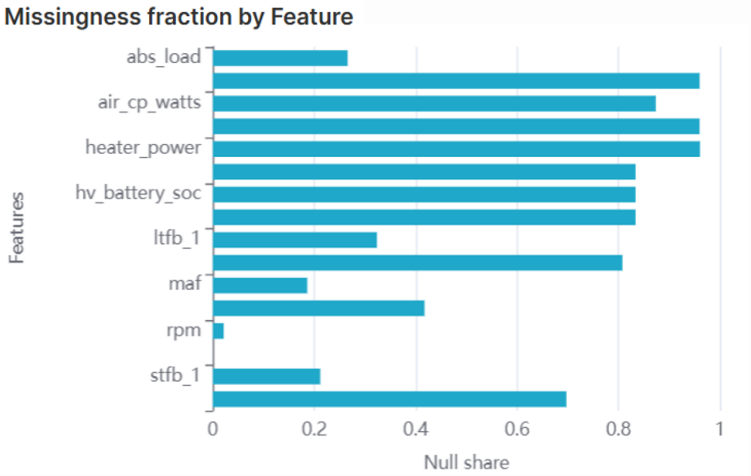
Stage 1. What we did:

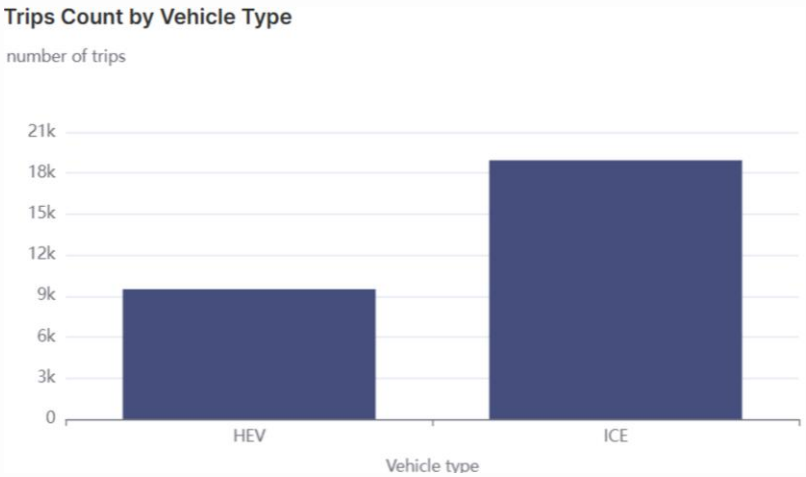
- Collected raw VED telemetry and vehicle metadata
- Created PostgreSQL tables: *vehicles* and *trips*
- Loaded source data into PostgreSQL
- Imported data to HDFS using Sqoop
- Stored warehouse data as Parquet + Snappy

Why?

- PostgreSQL gave us a structured relational source layer
- Sqoop was suitable for bulk transfer from PostgreSQL to HDFS
- Parquet + Snappy was chosen because the workload is read-heavy and column-oriented

Stage 2: Data Quality and Fleet Composition





Key problem:

- Direct *fuel_rate* is missing for about 96% of raw telemetry rows, so we cannot rely only on direct fuel measurements.
- Vehicle type imbalance must be considered.

From Sensor Rows to Trip-Level Features

What we did:

- Joined *trips* with *vehicles* by *vehid*.
- Estimated fuel rate using MAF when direct *fuel_rate* was missing.
- Aggregated sensor-level records into one row per (*vehid*, *tripid*).
- Created ML-ready trip-level features.

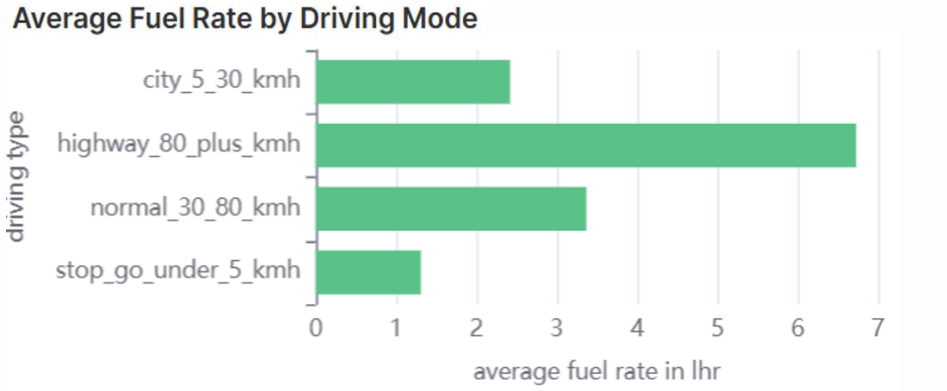
Why?

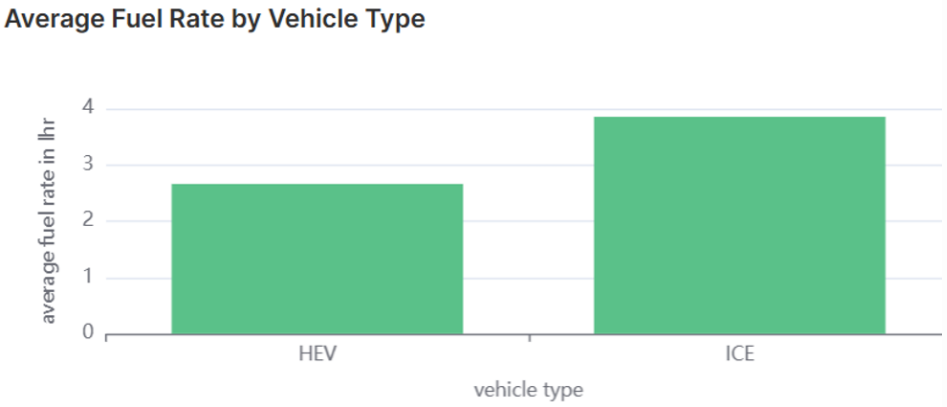
- ML target is trip-level fuel consumption, not timestamp-level sensor value.
- Vehicle metadata is needed because vehicle type, weight, engine displacement, and drivetrain affect fuel consumption.
- MAF approximation keeps more data usable despite sparse direct fuel measurements.

EDA: How this affects ML

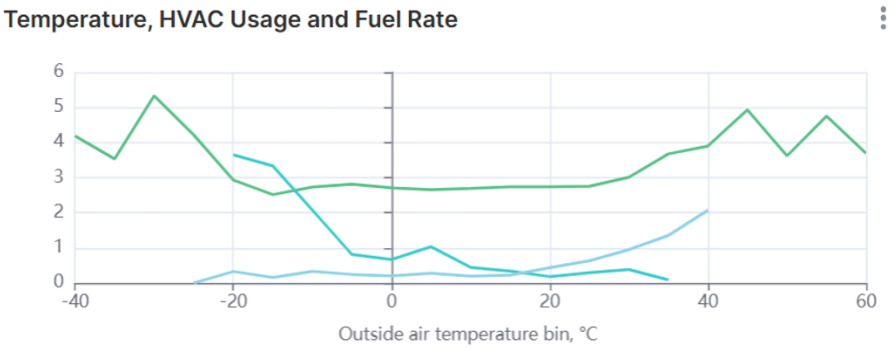
Findings:

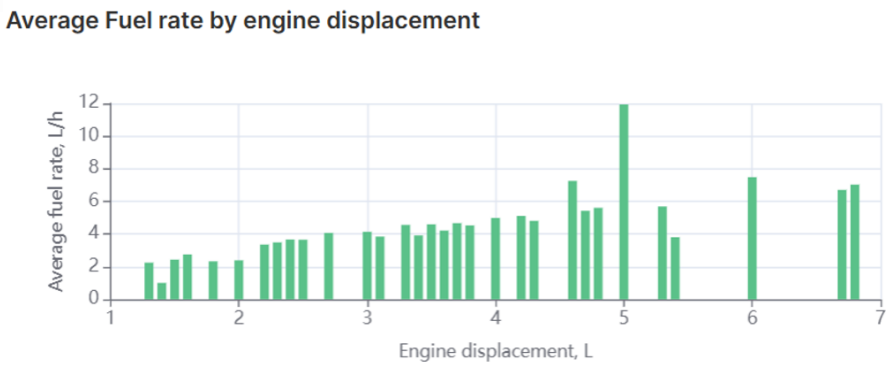
- Driving mode and vehicle type were kept as important explanatory features because fuel behavior differs across these groups.
- ICE vehicles consume more fuel on average than HEV vehicles.
- Driving mode and vehicle type are important ML features.





EDA Result: Feature Effects





Main observations:

- The relationship between fuel consumption and speed is nonlinear, so we tested tree-based models in addition to Linear Regression.
- Fuel rate increases with speed.
- Larger engines usually have higher fuel rate.
- Most fuel consumption values are in a realistic range.
- Extreme values should be filtered before ML.

Stage 3: Spark ML Modeling

ML task: predict *fuel_l_per_100km* for each trip.

Pipeline:

Clean trip-level features → outlier filtering → feature selection → train/test split →
→ Spark MLlib models → evaluation

Why these steps:

- Filtering removes unrealistic trips and target outliers.
- Mutual information was used because fuel consumption has nonlinear relationships with features.
- RMSE, MAE, and R^2 were used to evaluate both error size and explained variance.
- Linear Regression was used as a baseline. Random Forest and GBT were used because EDA showed nonlinear fuel behavior.

Best model	RMSE	MAE	R2
RandomForestRegressor	1.3968	0.7564	0.8882

Stage 4: Superset Dashboard

Dashboard structure:

- **Data Description** — explains raw data scale, schemas, and source tables.
- **EDA** — shows data quality and fuel-consumption patterns.
- **PDA** — presents ML results, best model, metrics, and selected features.

Why this structure:
It follows the project logic: first understand the data, then analyze patterns, then evaluate predictive modeling.

Challenges

Problem	Our decision	Why
Sparse fuel_rate	MAF-based approximation	Keeps more trips usable
Sensor-level rows	Trip-level aggregation	ML target is trip-level
Read-heavy analytics	Parquet + Snappy	Faster selected-column reads
Nonlinear fuel behavior	Tree-based models	Better fit for speed/mode interactions

Conclusions

Fuel consumption is strongly related to speed, driving mode, vehicle type, engine displacement, temperature, and HVAC usage

MAF-based approximation and trip-level aggregation made the data usable for analysis and forecasting

Missingness is the main data-quality limitation

Tree-based models performed better than a simple linear baseline

Thank you for attention!